Action Space Team 8: Defender Policy

Oct 24, 2026 · @Raghav

Overview

Our defender is a learned, shared policy that keeps attackers out of the site in 61.3% of simulated episodes (38.7% breach rate). Three DJI Tello defenders protect a site from two attacking drones in an 8 × 8 m room. An episode is a success when it ends with no attacker reaching the site.

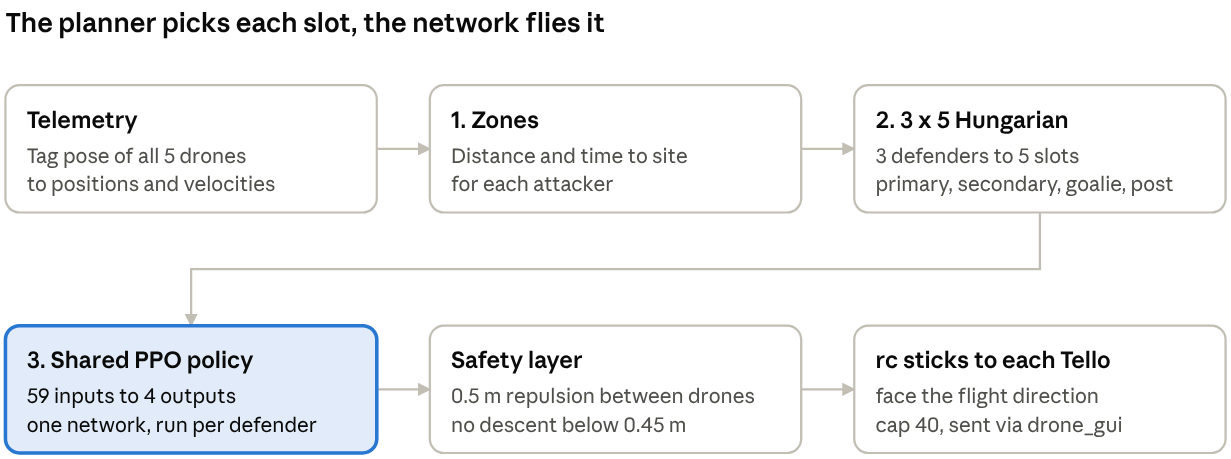
The approach has three steps, run 13.5 times a second:

1. **Zones.** Each attacker is placed in a zone by how far it is from the site and how soon it gets there.
2. **Assignment.** A 3 × 5 Hungarian assignment gives each defender one slot: primary or secondary on an attacker, goalie, or post.
3. **Flying the slot.** One shared neural network, trained with PPO, turns each defender's view and slot into a velocity command. A safety layer and a stick converter then drive the Tello.

The submission contains:

Item	Where
Code repository	`ActionSpaceTeam8/` (branch `defend-real`, full history in `ActionSpaceTeam8_history.bundle`)
Deployment script	`drone_deployment_infra/defender/run_defender.py`
Policy checkpoint	`checkpoints/shared_stage3_best.pt` (skrl PPO, 59 → 256 → 128 → 64 → 4)
This document	Design, training, results and how to run

System architecture



defender pipeline · one tick at 13.5 Hz

The planner decides who covers what; the network only decides how to fly there. A primary flies to the predicted intercept point and homes on the attacker within 1 m; a secondary holds a layered point behind it; the goalie guards the site; a defender with nothing to cover holds a post. The same planner code (`macro.py`) runs in training and at deployment.

Policy

One network is shared by all three defenders and runs once per defender per tick, deterministically (the mean action). Roles are not chosen by the network; they arrive as inputs from the planner.

Input: 59 numbers per defender (metres and m/s, world frame, relative to this defender unless stated)

Block	Size	Contents
Own state	6	Position relative to the site, own velocity
Planner target	3	Vector to the assigned point (the intercept point for a primary)
Base command	3	The planner's suggested velocity to that point

Block	Size	Contents
Slot	6	One-hot: post, primary A, primary B, secondary A, secondary B, goalie
Attackers (×2)	24	Per attacker: relative position (3), velocity (3), time to site (1, scaled 0 to 1), zone one-hot (3), alive (1), assigned to me (1)
Teammates (×2)	12	Per teammate: relative position (3), velocity (3); a defender that is out shows 1 km away
Last command	4	The command sent on the previous tick
Fix age	1	Age of this defender's own tag fix, in seconds

Network: 59 → 256 → 128 → 64 → 4, ELU hidden layers, tanh on the output. Inputs are normalised with the running mean and variance saved in the checkpoint:

```
\hat{x} = \mathrm{clip}\left(\frac{x - \mu}{\sqrt{\sigma^2} + 10^{-8}},\ -5,\ 5\right)
```

Output: 4 numbers in [−1, 1]: vx, vy, vz in the world frame (1.0 = 1 m/s) and a yaw rate. At deployment the yaw output is not used; the drone faces its direction of flight so the camera keeps seeing tags.

The deployment loader (`policy_runtime.py`) reproduces skrl's model exactly: on 4,096 random inputs the largest difference from skrl's own scaler and network was 0.0 for all three trained checkpoints.

Training

The policy was trained in Isaac Lab 2.3 (Isaac Sim 5.1) with skrl PPO on one H100, 2,048 parallel environments, task `AS-Defend-Role-v0`. A shared-policy wrapper presents the 3 defenders of every environment as separate samples of one policy; the critic also sees the true global state.

Curriculum. Each stage starts from the previous stage's best checkpoint.

Stage	What changes	Contact radius	Steps
1	Every defender is a primary; simpler attacker patterns, no dodging	0.35 m	6,000
2	Roles forced at random, so the network sees every slot	0.27 m	7,000
3	The Hungarian planner assigns the slots, as at deployment	0.2 m	9,000

Attackers fly straight, swerve, arc, feint, weave and jink, with altitude changes, speed bursts and reactive dodging. Defender response is modelled on the Tello (0.45 s response, 3 m/s² acceleration limit).

Rewards (per defender)

Term	Value	When
Breach	−10	Every defender, per attacker reaching the site
Success	+10	Every defender, episode ends with no breach
Team interception	+5	Every defender, per attacker downed
Contact	+10	The defender that made the contact
Primary progress	1.0 × metres gained	Toward the predicted intercept point (lead, not chase); the attacker itself within 1 m
Secondary progress	0.5 × metres gained	Toward its layered point behind the primary
Goalie and post progress	0.3 × metres gained	Toward their point
Secondary bonus	+5	When its attacker goes down
Goalie hold	+0.02 per step	Within 0.3 m of the goalie point
Self collision	−1 per step	Within 0.3 m of another defender
No tag visible	−0.02 per step	No mapped AprilTag in the camera image (room mode)

Term	Value	When
Floor	−2	Hitting the floor

PPO settings: tanh-bounded mean with no action clipping in the model (the environment clamps), KL early stop at 0.05, fixed learning rate 3e-4, discount 0.995. A guard resets any environment that produces non-finite values or speeds above 50 m/s and leaves it out of the statistics.

Contact rules, added later for evaluation: attrition (a contact can take the defender out) with a random outcome per contact, one third each: both down, attacker survives, attacker down. A further rule, single use, takes every defender that makes contact out of play. Both are switches in the config (`defender_attrition`, `defender_single_use`); `shared_stage3_best.pt` was trained before them.

Results

The submitted policy breaches the site in 38.7% of episodes under the stage 3 conditions it was trained for. Breach rate = share of episodes in which at least one attacker reaches the site; lower is better. Each figure is over 512 episodes (seed 1234).

Test	Breach rate
Stage 3 conditions (all attacker patterns, 0.2 m contact)	38.7%
Training mix, attrition on	42.4%
Aggressive attackers at equal speed, attrition on	17.3%
Faster attackers, attrition on	66.0%
Room (walls, tag-limited sensing), attrition on	67.8%

Attrition (a contact can also take the defender out) makes every test harder, and the room is the hardest case. A run trained from scratch with attrition on throughout scored worse on every test, so we kept `shared_stage3_best.pt`.

Deployment

`run_defender.py` runs the whole loop on a laptop CPU with drone_gui: it reads telemetry, runs the planner and the network, and posts rc sticks for each defender. Install `torch`, `numpy`, `pyyaml` and `scipy` (`defender/requirements.txt`), start drone_gui, then:

```
cd drone_deployment_infra/defender
python run_defender.py --defenders TELLO-1,TELLO-2,TELLO-3 --attackers TELLO-7,TELLO-8 \
    --policy ../../checkpoints/shared_stage3_best.pt --site 2.5,0,1.25
```

Option	Default	Use
`--policy`	required	Path to the trained checkpoint
`--site`	`2.5,0,1.25`	Protected site, world frame X,Y,Z in m
`--attacker-feed`	`gui`	`udp` to take attacker positions from an external feed (`--feed-port`, `--feed-frame map` or `world`)
`--map`	found automatically	Room tag map (`config/map.yaml`)
`--hz`	13.5	Control rate
`--rc-cap`, `--rc-step`	40, 15	Stick limit and largest change per tick
`--dry-run`	off	Compute and log, send no sticks
`--duration`	0 (until Ctrl+C)	Stop after this many seconds

Per tick: telemetry (each drone's tag pose) → world positions and velocities → zones and Hungarian slots → the 59 inputs → network → safety layer (0.5 m repulsion between defenders, no descent below 0.45 m) → rotate into each drone's frame → sticks a (right), b (forward), c (up), d (yaw to face the flight direction) → POST to drone_gui. If the script stops, drone_gui's deadman zeroes the sticks within 0.35 s. Each tick is logged to `outputs/defender_log.jsonl`.

Checks done:

- Policy parity: the deployment loader matches skrl's model with a largest difference of 0.0 on the three trained checkpoints.

- End to end without drones: `defender/tests/fake_gui.py` stands in for drone_gui (same HTTP and UDP formats, 3 defenders and 2 attackers that move on the sticks they receive). The script loaded the policy, assigned slots and sent sticks about 13 times a second to each defender, and none to the attackers.
- Frames: assigned defenders flown to their targets reached their attackers (0.0 m and 0.06 m), so pose and stick directions are mapped correctly.
- Unit tests: `defender/tests/test_policy_runtime.py` (5 tests) and `simulation_infra/tests/test_defend_real_macro.py` (18 tests, including Hungarian against brute force).

Repository and reproduction

All our work is in new files; the organisers' kit files are unchanged (our evaluator is a separate `evaluate_defend.py`, and the base defend environment is frozen as a copy inside our package).

Path	What it is
`drone_deployment_infra/defender/`	Deployment: `run_defender.py`, `policy_runtime.py`, `macro.py` (byte-identical to the sim's), tests, `fake_gui.py`
`simulation_infra/source/action_space_kit/action_space_kit/tasks/direct/tello_defend_real/`	Our environments (`AS-Defend-Real-v0`, `AS-Defend-Role-v0`), planner, shared-policy wrapper, PPO config
`simulation_infra/scripts/skrl/train_shared.py`	Training with the shared-policy wrapper
`simulation_infra/scripts/evaluate_defend.py`	Evaluation: `--shared`, `--checkpoint`, `--env_set`, `--seed`
`simulation_infra/tests/test_defend_real_macro.py`	Planner tests
`checkpoints/shared_stage3_best.pt`	The submitted policy
`ActionSpaceTeam8_history.bundle`	Full git history (`git clone ActionSpaceTeam8_history.bundle repo`)

Train (from `simulation_infra`, in the Isaac Lab environment), one stage at a time, each resuming from the last:

```
python scripts/skrl/train_shared.py --task AS-Defend-Role-v0 --agent
skrl_ppo_shared_cfg_entry_point \
    --headless --num_envs 2048 agent.trainer.timesteps=6000 env.stage=1
env.contact_radius=0.35
# stage 2: --checkpoint <stage 1 best> env.stage=2 env.contact_radius=0.27
agent.trainer.timesteps=7000
# stage 3: --checkpoint <stage 2 best> env.stage=3 env.contact_radius=0.2
agent.trainer.timesteps=9000
```

Evaluate:

```
python scripts/evaluate_defend.py --task AS-Defend-Role-v0 --headless --
num_envs 256 --episodes 512 \
    --seed 1234 --shared --checkpoint ../checkpoints/shared_stage3_best.pt --
env_set stage=3
# attrition on: add defender_attrition=True after --env_set
```

Limitations and next steps

- **The room is the weak case.** In the room with aggressive attackers the policy breached in 97% of episodes. Next: fine-tune `shared_stage3_best.pt` in room mode with attrition on.
- **Faster attackers are the main gap** (66.0% breach): the policy cannot catch what it cannot outrun, so meeting attackers further out is the lever.
- **Contact rules change the scores a lot.** With the random-outcome and single-use rules the policy scores much worse, so results are only comparable under one rule.
- **Infrastructure.** `run_defender.py` drives the drones through drone_gui. Running inside the organisers' swarm_ops setup needs a small adapter that reads attackers from `formation/drone<N>` on the bus and our own defenders' positions.
- **Sim to real.** The network has not yet flown real Tellos; first flights should use `--dry-run`, then low `--rc-cap`.